## Homework:

- Complete the following GD code for gradient descent
- Apply GD to minimize "x^2 + 2*y^2" from x0 =[-1, 0.1], get the points and plot using `plot_iterations` , what do you observe?
- Now apply GD to minimize the Rosenbrock function `(1 - x)**2 + 100 * (y - x**2)**2` , from x0 =[-1.25, 1.5], with step = 0.0015. What do you observe? What do you think is happening?

In [1]:
```
import sympy as sp
import matplotlib.pyplot as plt
import numpy as np
def GD(var1: str, var2: str, func: str, x0: list, step: float = 0.1, maxit: int = 20):
    """
    Gradient Descent algorithm to minimize a function
    """
    x1 = sp.symbols(var1)
    x2 = sp.symbols(var2)
    F0 = sp.sympify(func)
    F = sp.lambdify((x1, x2), F0, modules='numpy')
    [g1, g2] = [sp.lambdify((x1, x2), F0.diff(x1)), sp.lambdify((x1, x2), F0.diff(x2))]

    def grad(x):
        return np.array([g1(*x), g2(*x)])

    x0 = np.array(x0)
    points = [x0]  # collect the initial solution and subsequent solutions

    # Gradient descent iterations
    for i in range(maxit):
        gradient = grad(points[-1])

        # Stop if gradient norm is small enough
        if np.linalg.norm(gradient) < 0.01:
            print(f"Converged at iteration {i}, gradient norm: {np.linalg.norm(gradient)
            break

        # Update: x_new = x_old - step * gradient
        x_new = points[-1] - step * gradient
        points.append(x_new)

    return points
```

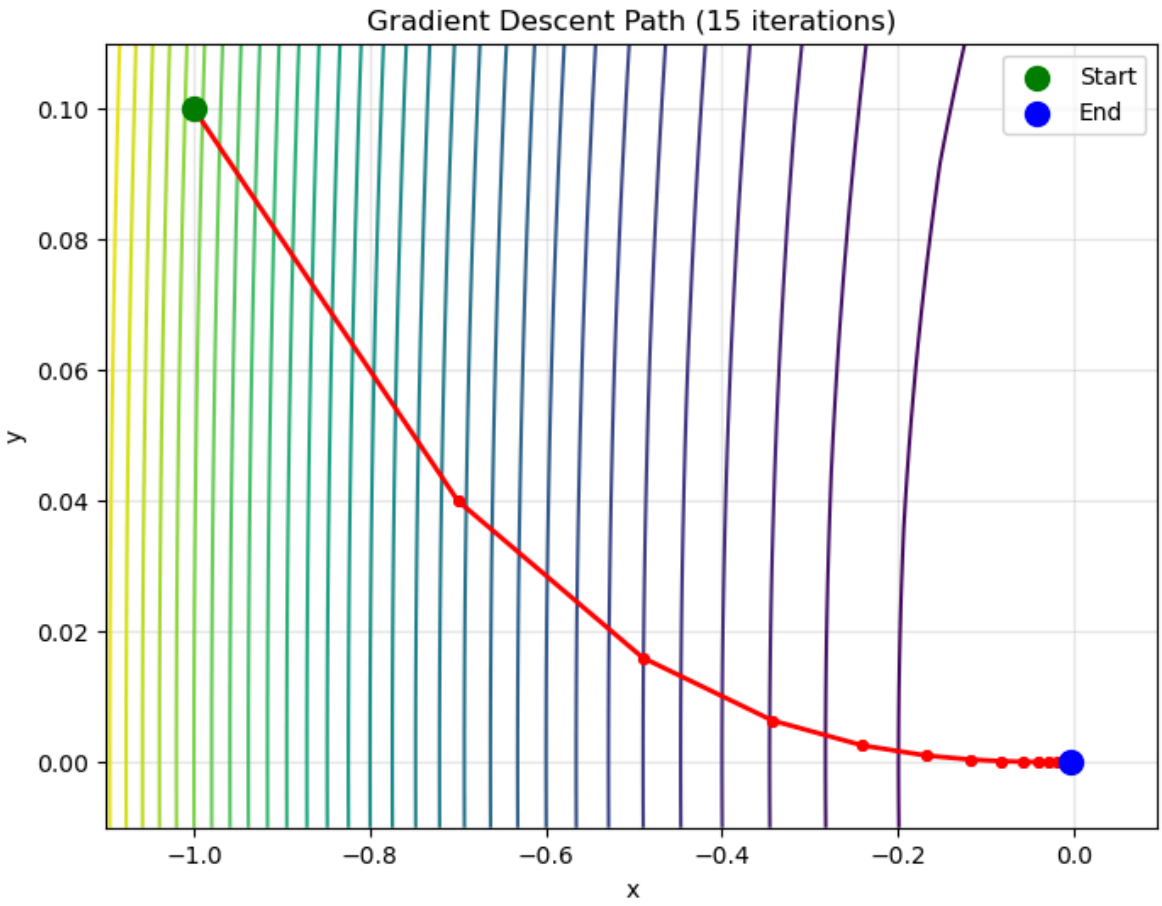
In [2]:
```
import sympy as sp
import matplotlib.pyplot as plt
import numpy as np
def plot_iterations(var1: str, var2: str, func: str, points: list, contours: int = 30,
    """
    Plot the optimization path on a contour plot
    """
    x1 = sp.symbols(var1)
    x2 = sp.symbols(var2)
    F0 = sp.sympify(func)
    F = sp.lambdify((x1, x2), F0, modules='numpy')

    def f(x):
        return F(*x)

    [xmax, ymax] = np.array(points).max(axis=0)
    [xmin, ymin] = np.array(points).min(axis=0)
    pad_x = (xmax - xmin) * 0.1
    xmin = xmin - pad_x
    xmax = xmax + pad_x
    pad_y = (ymax - ymin) * 0.1
    ymin = ymin - pad_y
    ymax = ymax + pad_y

    # Generate x and y values
    x = np.linspace(xmin, xmax, 30)
    y = np.linspace(ymin, ymax, 30)

    # Create a meshgrid
    X, Y = np.meshgrid(x, y)

    # Compute the function values
    Z = np.zeros_like(X)
    for i in range(X.shape[0]):
        for j in range(X.shape[1]):
            xx = X[i, j]
            yy = Y[i, j]
            Z[i, j] = f([xx, yy])

    fig, ax = plt.subplots(figsize=(8, 6))
    ax.contour(X, Y, Z, contours)
    plt.xlim([xmin, xmax])
    plt.ylim([ymin, ymax])

    # Plot starting point
    ax.scatter([points[0][0]], [points[0][1]], color='green', s=100, label='Start', zor

    # Plot path
    for i in range(len(points) - 1):
        ax.plot([points[i][0], points[i+1][0]], [points[i][1], points[i+1][1]],
                color='red', linewidth=2, marker='o', markersize=4)

    # Plot end point
    ax.scatter([points[-1][0]], [points[-1][1]], color='blue', s=100, label='End', zorc

    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_title(f'Gradient Descent Path ({len(points)-1} iterations)')
    ax.legend()
    plt.grid(alpha=0.3)
```

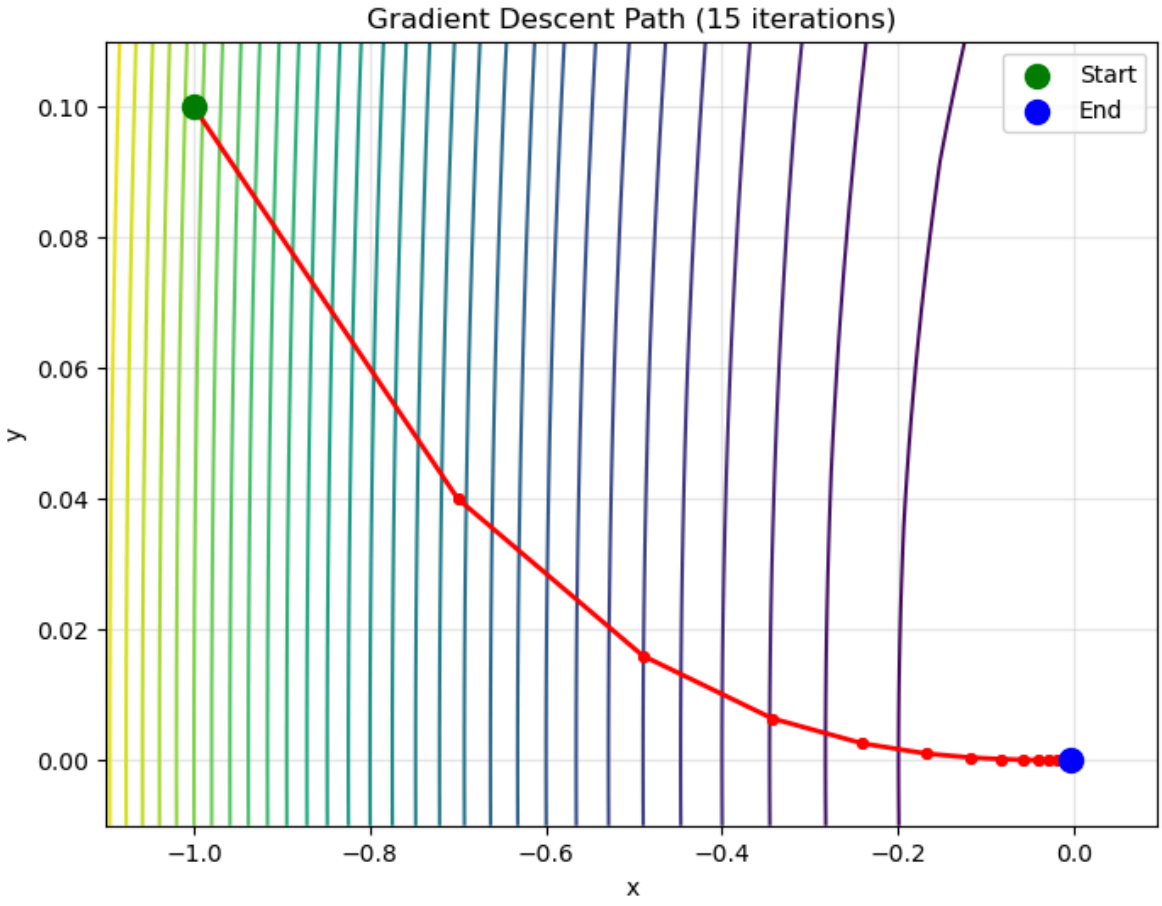
```
    return fig  # Return figure instead of showing
```

In [6]:
```
points = GD("x", "y", "x^2 + 2*y^2", x0 =[-1, 0.1], maxit = 100, step = 0.15)
plot_iterations("x", "y", "x^2 + 2*y^2", points,  contours = 30)
```

```
Converged at iteration 15, gradient norm: 0.009495
```

Out[6]:



- Add you observation

The gradient descent converges quickly and smoothly to the global minimum at (0, 0) in just 15 iterations. The function is convex with elliptical contours, so the optimization path follows a direct downhill trajectory. This demonstrates ideal behavior for gradient descent on a simple, well-conditioned quadratic function.

- Add your observation

The Rosenbrock function is much harder to optimize. After 1000 iterations, the algorithm only reaches approximately (0.50, 0.25), far from the true minimum at (1, 1). The optimization path shows characteristic "zigzagging" behavior as it slowly navigates a narrow, curved valley.